6.101 Midterm

Fall 2025

- You have 1 hour and 50 minutes to complete this exam. There are **7 problems**.
- **This exam is closed-book**.
  - You may use blank scratch paper.
  - You may **not** use any notes or cheat sheets. You may **not** use other electronics (including calculators, phones, tablets, music players, *etc.*) and you may **not** use other web sites or programs (including messaging platforms, search engines, LLMs, IDEs, documentation, editors, the Python interpreter, the course website, *etc.*).
  - Before the exam starts, **close all other applications, windows, and browser tabs on your laptop**, so that you can take the exam without any unintended interruptions. You should also **disable notifications** on your devices.
- Before you begin: you must **check in** by having the course staff scan the QR code at the top of the page.
- This page **automatically saves your answers** as you work. If you see a stuck yellow spinner, red exclamation mark, or a red notification that you are disconnected, your answers are not being saved: try reloading the page right away, before continuing to work on the exam.
- If you feel the need to **write a note to the grader**, you can click the gray pencil icon to the right of the answer.
- If you have a question, or need to use the restroom, please raise your hand.
- If you find yourself bogged down on one part of the exam, remember to keep going and work on other problems, and then come back.
- To **leave early**: enter *done* at the very bottom of the page and show your screen with the check-out code to a staff member.
- You **may not discuss** details of the exam with anyone other than course staff until grades have been released.

Good luck!

> Planning your time on the exam: **problems 1 and 2** are smaller, **problems 3 through 7** require up-front reading and are all building a single larger program.

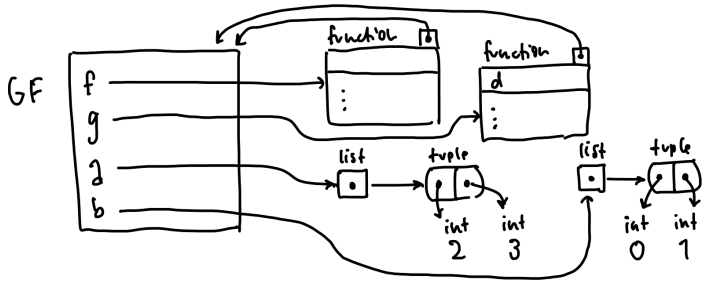
Problem 1

For each scenario below, fill in the implementation of function **`g(..)`** so that after all the code runs and garbage is collected, the state of the program matches the environment diagram ***—or—*** explain in one clear sentence why it is not possible to do so.

- Write **no more** than three lines of code. You may **not** use the `nonlocal` or `global` keywords (we'll learn about them later).
- For **full credit**, write no numbers other than `0`.

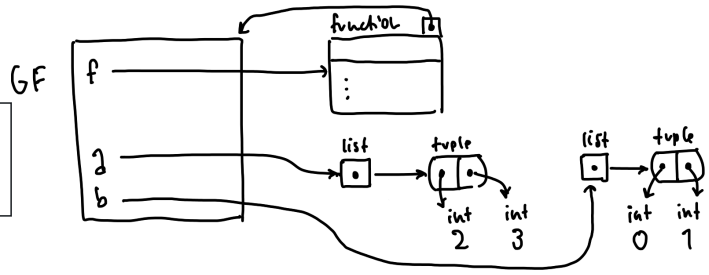
First scenario (note that the frames from calling `f` and `g` have been garbage-collected):

```
def f():
    a = [(0, 1)]
    b = [(2, 3)]
    b = g(a[0])
    return (a, b)

def g(d):
    
    
    
    

a, b = f()
```



Second scenario (again the frames from calling `f` and `g` have been garbage-collected):

```
def f():
    def g(d):
        
        
        
        
    a = [(0, 1)]
    b = [(2, 3)]
    b = g(a[0])
    return (a, b)

a, b = f()
```



Problem 2

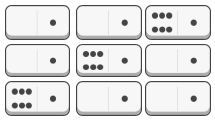
Domino pieces are rectangular game pieces divided into two squares, with zero to six dots (technical term: *pips*) on each square. We will represent dominoes as 2-element tuples of integers `0` through `6`.

Given these three dominoes:

```
dominoes = [(6, 1), (0, 1), (0, 1)]
```

We can arrange them into these three distinct sequences by re-ordering them, without rotating them:

```
sorted(all_sequences(dominoes)) # =>
        # [[(0, 1), (0, 1), (6, 1)],
        #  [(0, 1), (6, 1), (0, 1)],
        #  [(6, 1), (0, 1), (0, 1)]
```



Implement the function `all_sequences` **recursively** so it works as documented below.

(Suggestion: start by producing all permutations without considering the "distinct" requirement, then revise.)

```
def all_sequences(d):
    """
    Returns a list of all the distinct sequences of the dominoes in d (without
    rotating them), in any order, where each sequence is a list.
    >>> all_sequences([])
    [[]]
    >>> all_sequences([(1, 2)])
    [[(1, 2)]]
    >>> sorted(all_sequences([(1, 2), (3, 4), (5, 6)]))
    [[(1, 2), (3, 4), (5, 6)], [(1, 2), (5, 6), (3, 4)], [(3, 4), (1, 2), (5, 6)], \
     [(3, 4), (5, 6), (1, 2)], [(5, 6), (1, 2), (3, 4)], [(5, 6), (3, 4), (1, 2)]]
    >>> all_sequences([(1, 1), (1, 1), (1, 1)])
    [[(1, 1), (1, 1), (1, 1)]]
    """
```

Pips

You can **open this section in a new tab** so that you can refer to it more easily.

The *New York Times* recently introduced a new game called Pips. (Pips is edited by Ian Livengood. All images below © 2025 The New York Times Company.)

A **Pips puzzle** consists of a set of different domino pieces and an arbitrarily-shaped board.

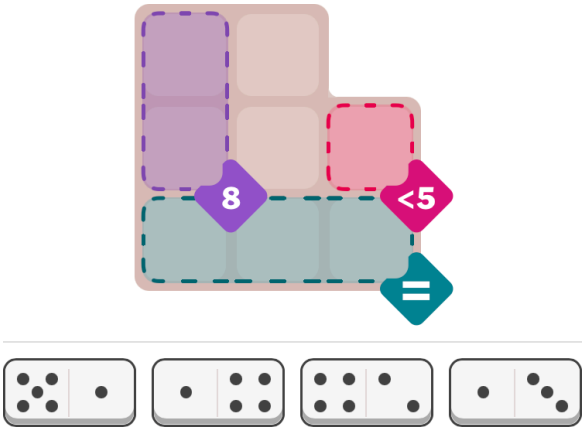
Regions of the board are annotated with condititons, for example:

- the sum of dots in that region must equal a certain number
- the sum of dots must be less than a certain number; or greater than a certain number
- all of the squares in the region must have the same number of dots; or must have different numbers of dots

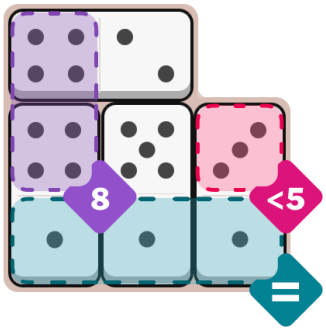
Some squares on the board might have no condition.

The goal is to place the dominoes to cover the board, satisfying the conditions.

For example, here is an “easy” puzzle:



And here is its solution:



We will represent a Pips puzzle using a dictionary with three keys:

```
puzzle = {
    "constraints": ...a dictonary...,
    "board": ...a dictonary...,
    "dominoes": ...a set...,
}
```

1. The `"constraints"` dictionary is a dictionary of strings to functions.

   The string keys are arbitrary, unique names for each region of the board that has a constraint.

   Each value, a function, takes a list of integers as input, corresponding to the number of dots in each square of the region. It returns `True` or `False` if the constraint on the region is or is not satisfied.

   (Note: we will change this in Problem 5 so that some of the input values may be `None`. For now, all inputs are `0` through `6`.)

   For our “easy” example, we could write:

   ```
   "constraints": {
       "makeEight": lambda nums: nums[0] + nums[1] == 8,
       "lessThanFive": lambda nums: nums[0] < 5,
       "all3Equal": lambda nums: nums[0] == nums[1] == nums[2],
   }
   ```

2. The `"board"` dictionary is a dictionary of 2-element tuples to strings or `None`.

   The keys of the dictionary are (row, column) coordinates, counting from the top left of the puzzle, down and to the right. The board shape is not constrained: coordinates with no board square are *omitted* from the dictionary.

   Each value in the dictionary is a string, indicating that square belongs to the constrained region with that unique name in the `"constraints"` dictionary; or `None`, indicating an unconstrained square.

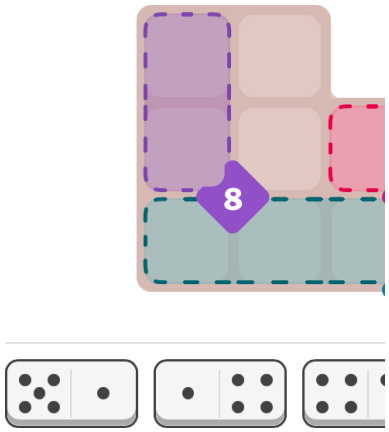
   ```
   "board": {
       (0, 0): "makeEight", (0, 1): None,        # no square (0, 2) on this board
       (1, 0): "makeEight", (1, 1): None,        (1, 2): "lessThanFive",
       (2, 0): "all3Equal", (2, 1): "all3Equal", (2, 2): "all3Equal",
   }
   ```

3. And the `"dominoes"` set is a set of 2-element tuples of integers 0 through 6 representing the available dominoes in their initial non-rotated orientation:

   ```
   "dominoes": {(5, 1), (1, 4), (4, 2), (1, 3)}
   ```

All together, here is our initial representation for the “easy” example:

```
easy = {
    "constraints": {
        "makeEight": lambda nums: nums[0] + nums[1] == 8,
        "lessThanFive": lambda nums: nums[0] < 5,
        "all3Equal": lambda nums: nums[0] == nums[1] == nums[2]
    },
    "board": {
        (0, 0): "makeEight", (0, 1): None,
        (1, 0): "makeEight", (1, 1): None, (1, 2): "lessThanFive",
        (2, 0): "all3Equal", (2, 1): "all3Equal", (2, 2): "all3Equal"
    },
    "dominoes": {(5, 1), (1, 4), (4, 2), (1, 3)}
}
```



To represent a **solution** to a puzzle, we use a dictionary where the non-rotated dominoes are the keys, and the values are 3-element tuples of integers:

- first the row and column position of the first listed square of the domino
- then a rotation value of `0`, `1`, `2`, or `3` that specifies where the second square of the domino is, relative to the first:

  `0` = to the right (domino is not rotated) *e.g.* 

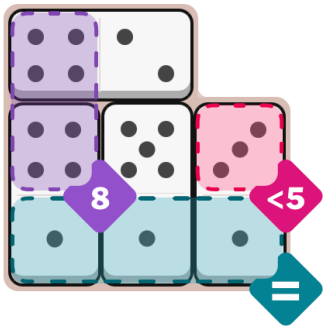
  `1` = below (domino is rotated 90° clockwise around the first listed square) 

  `2` = to the left (domino is rotated 180°) 

  `3` = above (domino is rotated 270°) 

For example:

```
easy_solution = {
    (5, 1): (1, 1, 1),
    (1, 4): (2, 0, 3),
    (4, 2): (0, 0, 0),
    (1, 3): (2, 2, 3),
}
```

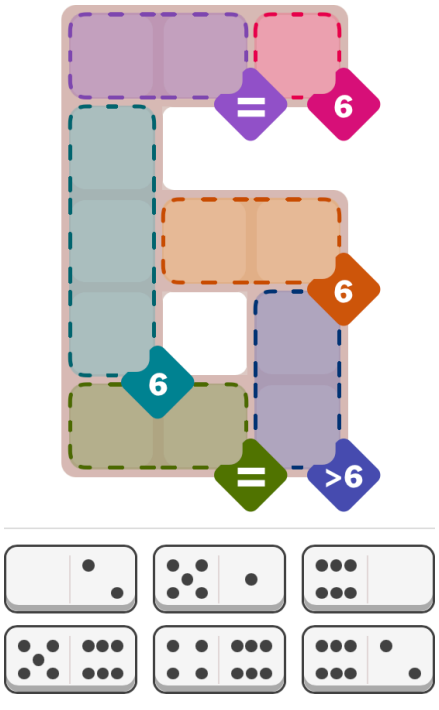


We will use the same format to represent partial solutions, where only some dominoes are placed:

```
easy_in_progress = {
    (5, 1): (1, 1, 1),
    (1, 4): (2, 0, 3),
}
```

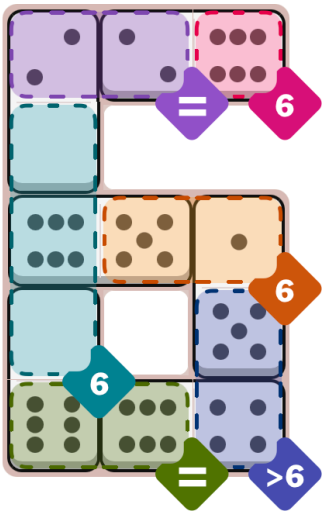
Finally, here is an example of a more complicated puzzle. You don't need to read it in detail, but notice how multiple regions can have identical constraints. Our representation gives different names (*e.g.* `"purple"` and `"green"`) to each region, even if their constraints are the same. The *Times* uses colors to help distinguish regions, and we have used those same colors as unique identifiers, but we could use any unique strings.

```
hard = {
    "constraints": {
        "purple": lambda nums: nums[0] == nums[1],
        "pink": lambda nums: nums[0] == 6,
        "aqua": lambda nums: nums[0] + nums[1] + nums[2] == 6,
        "orange": lambda nums: nums[0] + nums[1] == 6,
        "green": lambda nums: nums[0] == nums[1],
        "blue": lambda nums: nums[0] + nums[1] > 6
    },
    "board": {
        (0, 0): "purple", (0, 1): "purple", (0, 2): "pink",
        (1, 0): "aqua",
        (2, 0): "aqua",   (2, 1): "orange", (2, 2): "orange",
        (3, 0): "aqua",                     (3, 2): "blue",
        (4, 0): "green",  (4, 1): "green",  (4, 2): "blue",
    },
    "dominoes": {(0, 2), (5, 1), (0, 6), (5, 6), (4, 6), (6, 2)}
}
```



Here is the solution to the "hard" puzzle:

```
hard_solution = {
    (0, 2): (1, 0, 3),
    (5, 1): (3, 2, 3),
    (0, 6): (3, 0, 1),
    (5, 6): (2, 1, 2),
    (4, 6): (4, 2, 2),
    (6, 2): (0, 2, 2)
}
```



Let's build a program to solve these puzzles.

Reminder: you can **open this section in a new tab** so that you can refer to it more easily.

Problem 3

The constraint functions are going to be used by calling them with non-empty lists of integers, *e.g.*:

```
easy["constraints"]["makeEight"]([4, 4]) # => True
easy["constraints"]["lessThanFive"]([6]) # => False
```

Instead of writing lambdas for each one, constructing these constraint functions seems like something we could do with *higher-order functions*.

Implement the higher-order functions below so that in the definitions of our examples, we could write:

```
easy = {
    "constraints": {
        "makeEight": total_is(8),
        "lessThanFive": total_below(5),
        "all3Equal": all_same()
    },
    ...
```

... and:

```
hard = {
    "constraints": {
        "purple": all_same(),
        "pink": total_is(6),
        "aqua": total_is(6),
        "orange": total_is(6),
        "green": all_same(),
        "blue": total_above(6)
    },
    ...
```

Notice that "hard" regions `"pink"`, `"aqua"`, and `"orange"` all use `total_is(6)` even though they have different numbers of squares. Make sure all your functions will work accordingly.

Note: for now, constraint functions are called with lists of one or more integers `0` through `6`. We will re-implement these functions later after adjusting how they work in Problem 5.

```
def total_is(total):
```

```
def all_same():
```

(You do not need to implement the other functions, `total_below` and `total_above`.)

Problem 4

A solution to the puzzle will have three properties:

- it will use the correct set of dominoes
- the dominoes will exactly cover the board
- all the region constraints will be satisfied

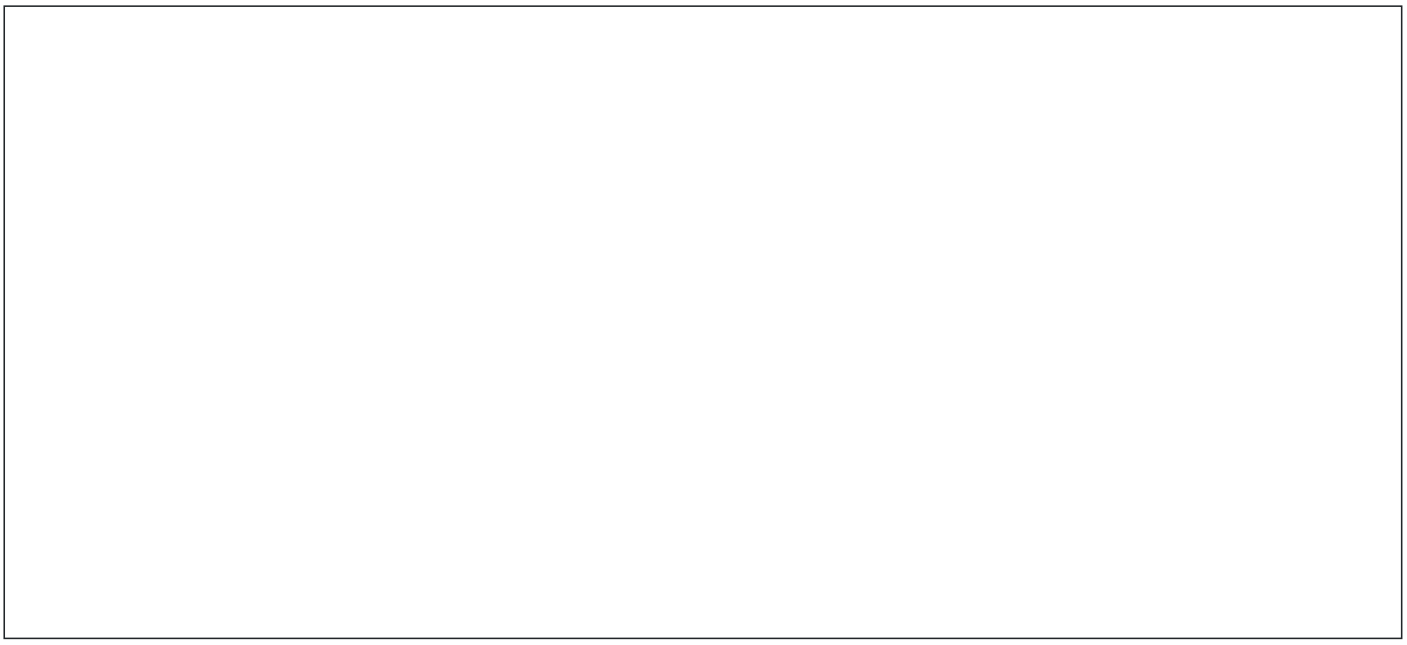
Consider this function to test whether a `solution` exactly covers the board of `puzzle`, using the correct dominoes:

```
def solution_covers_board(puzzle, solution):
    if puzzle["dominoes"] != solution.keys():
        return False
    expanded = expand_solution(solution)
    if expanded is None:
        return False
    return puzzle["board"].keys() == expanded.keys()
```

Implement **`expand_solution`** so that it works in `solution_covers_board` and as described below. As you read the doctests, note that the tests are comparing the return values for equality: `expand_solution` itself does not return `True` or `False`.

(Suggestion: use an idea similar to Snekoban's `DIRECTION_VECTOR` to help with rotation numbers.)

```
def expand_solution(solution):
    """
    Given a solution dictionary (mapping dominoes to their locations & orientations),
    returns a dictionary whose keys are all the (row, column) board squares covered
    and the values are the number of dots in those squares. If the solution specifies
    overlapping dominoes or squares at negative coordinates, returns None.
    >>> expand_solution(easy_solution) == {
    ...     (0, 0): 4,  (0, 1): 2,
    ...     (1, 0): 4,  (1, 1): 5,  (1, 2): 3,
    ...     (2, 0): 1,  (2, 1): 1,  (2, 2): 1}
    True
    >>> expand_solution({}) == {}
    True
    >>> expand_solution({(5, 1): (1, 1, 1)}) == {
    ...     (1, 1): 5,
    ...     (2, 1): 1}
    True
    >>> expand_solution({(5, 1): (1, 1, 1),
    ...                  (1, 4): (2, 0, 0)}) is None
    True
    """
```



Problem 5

Before we write a `solution_satisfies_constraints` to go along with `solution_covers_board`, we need to adjust our constraint functions. Instead of taking a list of only integers, some of the list elements might be `None`, representing un-filled squares where we have not *yet* placed a domino. The constraint function should return `True` if the constraint is **or could be** satisfied.

From the "easy" example:

```
total_is(8)([4]) # => False
total_is(8)([4, None]) # => True (the second square could be 4)
total_is(8)([4, 4]) # => True
total_is(8)([4, 3]) # => False (the total is 7)
total_is(8)([4, 5, None]) # => False (the total will be at least 9)

total_below(5)([4]) # => True
total_below(5)([4, None]) # => True
total_below(5)([4, 1]) # => False
total_below(5)([4, 1, None]) # => False

all_same()([1, 1, None, None]) # => True
```

Reimplement `total_is` with this new specification:

```
def total_is(total):
```

(You do not need to reimplement the other functions.)

Problem 6

Assume we have updated all the constraint functions in our puzzle representations with the revised behavior. Complete the implementation of **`solution_satisfies_constraints`**:

(Suggestion: given dictionary `expanded`, `expanded[c]` raises a `KeyError` when `c` is not a key, but `expanded.get(c)` returns `None` in that case instead.)

```
def solution_satisfies_constraints(puzzle, solution):
    """
    Given a puzzle dictionary and a solution dictionary, returns True if solution is
    consistent with the puzzle board (no dominoes placed outside the board) and
    constraints (all are satisfied or could be satisfied). Returns False otherwise.
    >>> solution_satisfies_constraints(easy, easy_solution)    # complete solution
    True
    >>> solution_satisfies_constraints(easy, easy_in_progress) # partial solution
    True
    >>> solution_satisfies_constraints(easy, {})               # no progress
    True
    >>> solution_satisfies_constraints(easy, {(5, 1): (1, 2, 1)}) # violates lessThanFive
    False
    >>> solution_satisfies_constraints(easy, {(1, 4): (2, 0, 1)}) # outside the board
    False
    """
    expanded = expand_solution(solution)
    if expanded is None:
        return False
    if expanded.keys() - puzzle["board"].keys():   # ← see hint & question about
        return False                               #   this line below
```

Explain in one clear sentence: what is the purpose of checking **`if`** `expanded.keys() - puzzle["board"].keys()`? (*Hint:* the " – " operator is computing a *set difference*: elements in the first set that are not in the second set.)

Problem 7

Finally, let's solve some puzzles. We have these pieces implemented above:

- `expand_solution(solution)` *# Given dict of domino to (row, col, rotation),*
  # return dict of (row, col) to dots -or- None if overlapping
- `solution_covers_board(puzzle, solution)` *# Return whether (possibly partial) solution*
  # exactly covers puzzle board
- `solution_satisfies_constraints(puzzle, solution)` *# Return whether (possibly partial)*
  # solution is consistent with constraints

Consider the following implementation of **`solve`**, which uses `find_path` from the reading; the code for `find_path` is included below for reference.

```
def solve(puzzle):
    """
    Returns a solution to the puzzle, or None if it cannot be solved.
    >>> solve(easy) == easy_solution
    True
    >>> solve(hard) == hard_solution
    True
    """
    rows = max(row for (row, col) in puzzle["board"]) + 1
    cols = max(col for (row, col) in puzzle["board"]) + 1

    def solution_to_state(solution):
        """Represent a solution as a tuple of sorted (key, value) dict item tuples."""
        return tuple(sorted(solution.items()))

    def state_to_solution(state):
        """Re-create a solution dict from its tuple representation."""
        return {domino: placement for (domino, placement) in state}

    def neighbors(state):
        solution = state_to_solution(state)
        a = []
        for b in puzzle["dominoes"]:
            for c in range(rows):
                for d in range(cols):
                    for e in range(3):
                        solution[b] = (c, d, e)
                        a.append(solution)
        return a

    def goal(state):
        solution = state_to_solution(state)
        return (solution_covers_board(puzzle, solution) and
                solution_satisfies_constraints(puzzle, solution))

    start = solution_to_state({})
    path = find_path(neighbors, start, goal)
    return state_to_solution(path[-1]) if path else None
```

Most of this code is good! But the implementation of `neighbors` is unreadable, has some bugs, and (if the bugs were fixed) results in a `solve(..)` that is very inefficient. Write a new implementation that is clear, correct, and much more efficient:

```
def neighbors(state):
    solution = state_to_solution(state)
```

Explain in one clear sentence (*no Python !*) for each question:

What is the purpose of `solution_to_state` and `state_to_solution`?

The starting state is created from the empty partial solution by the line `start = solution_to_state({})`.
In general, in your implementation, what are the neighbors of a state?

And in particular, in your implementation, what are the neighbors of the start state?

find_path

For reference:

```
def find_path(neighbors_function, start, goal_test):
    """
    Find a path through a state graph defined by `neighbors_function`, starting from
    the `start` state and reaching a state satisfying `goal_test`.

    neighbors_function: function that takes a state and returns its neighbors
                        (as an iterable)
    start: starting state
    goal_test: function that takes a state and returns True (or truthy) if and only
               if the state satisfies the goal condition

    Returns the path of states from start to a goal state, or None if no path exists.
    """
    if goal_test(start):
        return (start, )

    agenda = [(start, )]
    visited = {start}

    while agenda:
        this_path = agenda.pop(0)
        terminal_state = this_path[-1]

        for neighbor in neighbors_function(terminal_state):
            if neighbor not in visited:
                new_path = this_path + (neighbor,)

                if goal_test(neighbor):
                    return new_path

                agenda.append(new_path)
                visited.add(neighbor)

    return None
```